

Research Note: Reimagining Low-Level Design Practice

Research Note: Reimagining Low-Level Design (LLD) Practice

Author: Aryaman Malik

Topic: Learner Problems, Market Landscape Analysis, and Product Direction for LLD Practice

Date: September 2026

1. The Learner Problem: Why Practicing LLD is Uniquely Hard

Low-Level Design (Object-Oriented Design / Machine Coding) is an essential evaluation stage for mid-to-senior software engineers. Unlike Data Structures & Algorithms (DSA), which typically has binary correctness (passes test cases within time/space limits), LLD is inherently open-ended and subjective:

1. **"Multiple Valid Designs" Dilemma**: A Parking Lot can be modeled with a centralized manager, a distributed event-driven gate model, or a stateful slot matrix. A learner seeing a solution different from their own cannot easily tell whether their solution is flawed, equally valid with different trade-offs, or actually superior for certain constraints.
2. **Lack of Concrete Evidence in Feedback**: Learners frequently encounter hand-wavy reviews such as "this lacks abstraction" or "doesn't follow SOLID". Without concrete citations linking design smells to specific classes, interfaces, or method signatures, learners cannot pinpoint what to refactor.
3. **No Closed-Loop Learning**: Today, learners practice by sketching a class diagram on a whiteboard or writing a code file, glancing at an online reference blog post, nodding, and moving to the next problem. They rarely attempt the *same* problem a second time to see whether their revisions resolved coupling or testability issues.
4. **Artifact Ambiguity**: What does a learner actually need to provide for a design to be meaningful? Writing 1,000 lines of boilerplate getters/setters proves nothing about design thought; conversely, drawing three boxes with no method contracts fails to demonstrate interface segregation or concurrency awareness.

2. Research of Existing Approaches & Platforms

We analyzed the current ecosystem of engineering preparation and design evaluation tools:

| Platform / Approach | Primary Format | Evaluation Model | Core Strengths | Critical Gaps |

| :--- | :--- | :--- | :--- | :--- |

| LeetCode / HackerRank | Concrete Code | Deterministic unit tests & I/O execution | Automated, instantaneous, zero ambiguity on algorithmic correctness. | Optimized for algorithmic puzzles. No concept of class responsibilities, extensibility, design patterns, or coupling. |

| GitHub Repositories (e.g. Awesome-Low-Level-Design) | Static Code & Readmes | None (Self-audit against reference solutions) | High variety of community-contributed problem solutions. | Single "golden" solution creates false dogma; provides zero feedback on learner's own alternative ideas. |

| Course Platforms (Educative, NeetCode, SystemDesignSchool) | Articles & Video Walkthroughs | Multiple choice quizzes / Passive reading | High-quality pedagogical breakdown of patterns and canonical designs. | Completely passive consumption. The learner does not actually practice design synthesis under constraints. |

| Human Mock Interviews (Pramp, Interviewing.io) | Interactive Dialogue + Shared Doc | Human senior engineer commentary | Deep qualitative insights, dynamic follow-up requirements, nuanced trade-off discussions. | Expensive (\$150-\$250/hr), high scheduling friction, inconsistent interviewer rubrics, impossible to practice daily in an agile loop. |

3. Key Gaps Identified

From the synthesis above, four critical product gaps emerge:

1. ****Gap 1: Absence of Standardized, Explainable Rubrics****

Learners do not need a single 0–100 vanity score. They need multi-dimensional evaluation: Requirements Understanding, Single Responsibility (SRP), Coupling & Encapsulation, Interface Abstractions & Extensibility, and Edge Case / Concurrency Handling.

2. ****Gap 2: Disconnect Between Feedback and Concrete Evidence****

Useful feedback must point directly to what the user wrote: criterion -> score -> evidence -> concern -> suggestion -> confidence. If an evaluator flags high coupling, it must cite: "Class ParkingLot directly instantiates CreditCardProcessor inside unparkVehicle()".

3. ****Gap 3: All-or-Nothing Evaluation Pipeline****

Many AI-driven prototypes make the mistake of dumping unstructured user text into an LLM with "Grade this design". This yields hallucinations, inconsistent scores, and zero structural guarantees. Conversely, pure rule-checkers fail to evaluate architectural wisdom. A hybrid pipeline (Deterministic Pre-validation + Structured Rubric Reasoning) is required.

4. ****Gap 4: No Attempt Progression Tracking****

Platforms treat each submission as an isolated event. Learners need to see: "In Attempt 1, your Coupling score was 2/5 due to tight dependency on payment modes. In Attempt 2, you introduced a PaymentStrategy interface, raising your Extensibility to 5/5."

4. Product Direction & MVP Focus

To solve these gaps without over-engineering an unmanageable LMS, our MVP concentrates strictly on the core learner practice loop:

\$\$\text{Choose Problem} \rightarrow \text{Think \& Design} \rightarrow \text{Submit} \rightarrow \text{Structured Feedback} \rightarrow \text{Review \& Compare} \rightarrow \text{Iterate}\$\$\$

Strategic Design Choices for the MVP:

- ****Submission Format****: A structured multi-faceted design canvas comprising:
 1. ***Requirements & Assumptions*** (functional & non-functional scope)
 2. ***Entities, Interfaces & Responsibilities*** (classes, methods, relationships)
 3. ***Patterns, Extensibility & Trade-offs*** (justifications for design choices)
 4. ***Visual / Structural Diagram*** (Mermaid class diagram preview or code skeleton)
- ****Evaluation Pipeline****:
 - ****Deterministic Gatekeeper****: Validates schema, required sections, entity definitions, and contract declarations.
 - ****Explainable Rubric Evaluator****: Applies a strict 5-dimensional rubric yielding evidence-backed feedback with constructive suggestions. Built with a heuristic semantic engine (and pluggable LLM adapter for live Gemini inference).
- ****Attempt History & Evolution****:
 - Learners can preserve multiple attempts per problem and compare scores and feedback across iterations to see measurable improvement.
- ****Fail-Safe & Idempotent Architecture****:
 - Submissions are recorded immediately upon receipt. Asynchronous evaluation transitions (``SUBMITTED`` $\rightarrow$ ``EVALUATING`` $\rightarrow$ ``COMPLETED`` / ``FAILED``) guarantee learner work is never lost. Idempotency keys prevent duplicate runs on network retries.